

Cuantización, PEFT, y despliegue de Modelos de Lenguaje Grandes

Transformers, uso de agentes, LangChain y RAG

Nicolás Cayturo



Large Language Models (LLMs)

Motivación

La utilización de modelos de lenguaje basados en Transformers ha revolucionado el campo del Procesamiento de Lenguaje Natural (NLP) debido a su modularidad para ser adaptado a múltiples tareas, pero se sabe que el tamaño del modelo afecta su rendimiento.

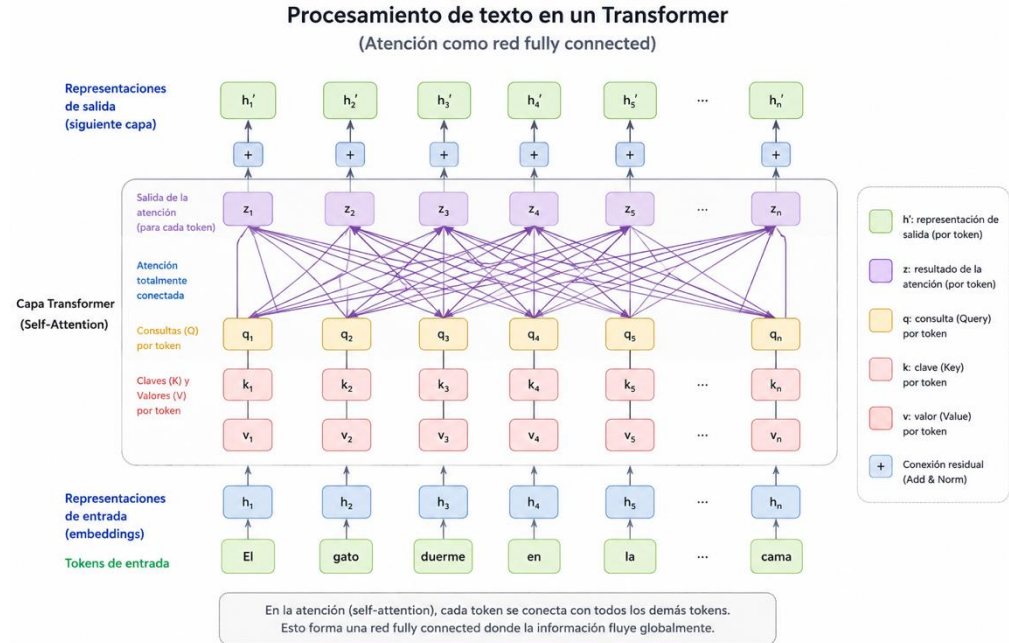
Los grandes modelos de lenguaje tienen órdenes de magnitud más parámetros que los modelos pre entrenados anteriores y se ha observado que presentan habilidades extraordinarias:

- Aprendizaje en contexto.
- Seguimiento de instrucciones.
- Razonamiento paso a paso.
- Generación flexible. Greedy search, beam search, sampling y temperatura.

Modelos basados en Transformers

El problema del “olvido” o de la información distante y la naturaleza secuencial de las redes neuronales recurrentes (RNN) llevaron al desarrollo de los Transformers; un acercamiento al procesamiento de secuencias que elimina las conexiones recurrentes y se asemeja más a las redes totalmente conectadas.

Todos los modelos de NLP del estado del arte están basados en Transformers.



Habilidades emergentes

Habilidad	Hito	¿Por qué surgió?	Paper
In-context learning	2020	Para <i>adaptar</i> un modelo a tareas nuevas mediante instrucciones y ejemplos sin necesidad de reentrenamiento.	https://arxiv.org/pdf/2005.14165 <i>Language models are few-shot learners</i> OpenAI
Cadena de razonamiento (chain-of-thought)	2021-2022	Para mejorar problemas que requieren cálculos o inferencias intermedias.	https://arxiv.org/pdf/2201.11903 <i>Chain-of-thought prompting elicits reasoning in LLMs</i> Google Research
Seguimiento de instrucciones complejas	2021-2022	Para alinear respuestas del modelo con la intención y restricciones del usuario.	https://arxiv.org/pdf/2203.02155 <i>Training language models to follow instructions with human feedback</i> OpenAI
Generación de código funcional	2021	Transformar instrucciones en lenguaje natural a bloques de código funcionales (ejecutable, verificable).	https://arxiv.org/pdf/2107.03374 <i>Evaluating LLMs trained on code</i> https://arxiv.org/pdf/2108.07732 <i>Program synthesis with LLMs</i>

Habilidades emergentes

In-context learning

Capacidad de un modelo de lenguaje para **inferir** cómo realizar una tarea (por ejemplo, traducción, clasificación, respuesta a preguntas, etc.) a partir de instrucciones o ejemplos incluidos en el prompt, sin actualizar sus parámetros o reentrenar la red.

- Zero-shot: solo se proporciona la instrucción.
- One-shot: se proporciona un ejemplo.
- Few-shot: se proporcionan varios ejemplos.

A pesar de que los grandes modelos de lenguaje en sí presentan buenas capacidades de aprendizaje en contexto, algunas técnicas como el *afinamiento supervisado de instrucciones* se utilizan para mejorar las capacidades de aprendizaje en contexto al ajustar los parámetros del modelo entrenándolo en tareas de seguimiento de instrucciones.

Habilidades emergentes

Seguimiento de instrucciones

Capacidad de un modelo entrenado para seguir instrucciones en tareas no antes vistas que también están descritas en forma de instrucciones.

Por ejemplo, elabora una ficha de evaluación de una herramienta de IA aplicada a la educación. Debes cumplir con las siguientes condiciones:

- Selecciona una herramienta de IA generativa.
- Describe su finalidad en máximo de 40 palabras.
- Incluye exactamente 4 criterios de evaluación.
- Presenta el resultado en una tabla.
- Añade una recomendación de máximo 50 palabras.
- ...

Instruction Tuning

In-context exemplars not needed to learn the task

Instruction
Exemplar
Label

Instruction
Exemplar
Label

Evaluation
Example

Input

What is the sentiment of this?

This movie is great

Answer: Positive

Relevant

What is the sentiment of this?

Worst film I've ever seen

Answer: Negative

Relevant

[more exemplars]

What is the sentiment of this?

This movie is terrible

Answer:

Output

Negative

Habilidades emergentes

Cadena de razonamiento

Habilidad del modelo que le permite resolver problemas complejos utilizando razonamientos intermedios. Los primeros modelos solían responder de manera directa.

Standard Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ❌

Chain-of-Thought Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✅

Habilidades emergentes

Generación de código funcional

Capacidad del modelo para generar código que no solo sea sintácticamente correcto, sino que también:

- Ejecute la tarea solicitada
- Maneja casos borde
- Valide entradas
- Produzca el resultado esperado
- Pueda verificarse mediante pruebas (tests generados por humanos)

prompt

Write a python function to check if a given number is one less than twice its reverse. Your code should satisfy these tests:

```
assert check(70) == False
assert check(23) == False
assert check(73) == True
```

model

```
def check(n):
    if n == 2*int(str(n)[::-1])-1:
        return True
    else:
        return False
```

prompt

Write a function to find the smallest missing element in a sorted array. Your code should satisfy these tests:

```
assert smallest_missing([0, 1, 2, 3, 4, 5, 6], 0, 6) == 7
assert smallest_missing([0, 1, 2, 6, 9, 11, 15], 0, 6) == 3
assert smallest_missing([1, 2, 3, 4, 6, 9, 11, 15], 0, 7) == 0
```

model

```
def smallest_missing(arr, n, m):
    smallest = min(n, m)
    for i in range(n, m + 1):
        if arr[i] <= smallest:
            smallest += 1
    return smallest
```

prompt

Write a Python function to sort the given array by using merge sort. Your code should satisfy these tests:

```
assert merge_sort([3, 4, 2, 6, 5, 7, 1, 9]) == [1, 2, 3, 4, 5, 6, 7, 9]
assert merge_sort([7, 25, 45, 78, 11, 33, 19]) == [7, 11, 19, 25, 33, 45, 78]
assert merge_sort([3, 1, 4, 9, 8]) == [1, 3, 4, 8, 9]
```

model

```
def merge_sort(arr):
    if len(arr) < 2:
        return arr;
    mid = len(arr) // 2;
    left = arr[:mid];
    right = arr[mid:];
    left = merge_sort(left);
    right = merge_sort(right);
    merged = [];
    i = j = 0;
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            merged.append(left[i]);
            i += 1;
        else:
            merged.append(right[j]);
            j += 1;
    merged.extend(left[i:]);
    merged.extend(right[j:]);
    return merged;
```

Prompting

Cómo especificamos una tarea a un LLM

Los LLMs están entrenados principalmente para predecir el siguiente token. El prompting convierte una tarea concreta (clasificar, resumir, traducir o razonar) en una secuencia de texto que el modelo sea capaz de completar.

El prompting **aparece después de las habilidades emergentes porque estas no se manifiestan de la misma manera ante cualquier entrada**. El usuario debe presentar la tarea mediante un prompt.

Capacidad del modelo + Instrucción del usuario = Respuesta a una tarea concreta

Por ejemplo: el modelo puede tener capacidad para resumir pero necesita una instrucción:

“Resume el siguiente artículo en 100 palabras y destaca el objetivo y los resultados”.

Prompting

Un prompt convierte una necesidad “humana” en una tarea comprensible para el modelo. Puede contener: objetivo, contexto, datos de entrada, ejemplos, restricciones, formato de salida.

Por ejemplos: *Completa la siguiente oración y clasifica el sentimiento de esta.*

Amé esa película. En general, la película me pareció [Z].

El modelo busca palabras para completar la oración:

- Buena, Mala, Terrible, Bacán

Finalmente, esas palabras se convierten en una categoría:

Buena, bacán → sentimiento *positivo*

Mala, terrible → sentimiento *negativo*

Instruction tuning

A diferencia del prompting, el ajuste de instrucciones ocurre durante el entrenamiento.

El modelo se entrena previamente con muchos ejemplos de:

- Instrucción
- Entrada
- Respuesta/salida esperada

Por ejemplo:

- Instrucción: Traduce al inglés
- Entrada: Buenos días.
- Salida: Good morning.

Durante el uso, un usuario podría enviar una frase distinta, y aun así el modelo puede generar la salida esperada.

Example task instances

Instance

- **Input:** Sentence: It's hail crackled across the comm, and Tara spun to retake her seat at the helm.
- **Expected Output:** How long was the storm?

Instance

- **Input:** Sentence: There was even a tiny room in the back of one of the closets.
- **Expected Output:** After buying the house, how long did it take the owners to notice the room?

Instance

- **Input:** Sentence: During breakfast one morning, he seemed lost in thought and ignored his food.
- **Expected Output:** How long was he lost in thoughts?

Plantilla de instrucciones

Las distintas familias de modelos pueden utilizar formatos o plantillas de conversación distintos.

<https://developers.openai.com/api/docs/guides/prompt-engineering>

https://platform.claude.com/docs/en/claude_api_primer

La plantilla de instrucciones convierte una conversación como esta:

Sistema: Eres un tutor de agentes.

Usuario: Explica qué es un agente.

Asistente: ...

Es importante porque en un agente pueden existir: sistema, usuario, asistente, llamada a herramienta, resultado de herramienta, asistente.

En una secuencia de tokens especiales que el modelo reconoce:

<TOKEN_SISTEMA>

Eres un tutor de programación.

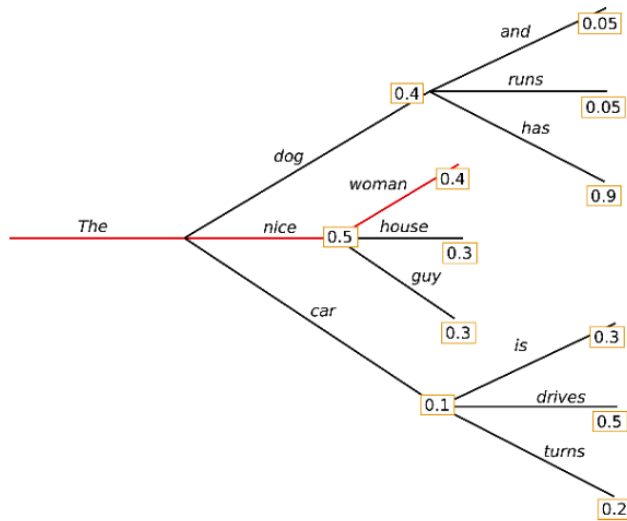
<TOKEN_USUARIO>

Explica qué es una función.

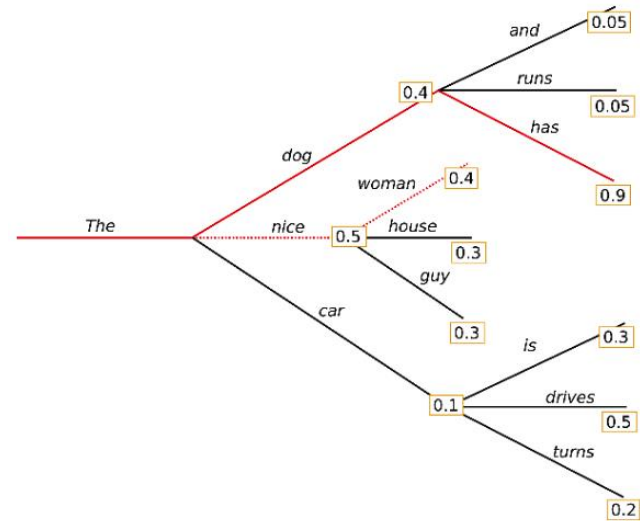
<TOKEN_ASISTENTE>

Técnicas de generación de la respuesta

Greedy search: en cada paso se predice la palabra con mayor probabilidad. Las secuencias comienzan a repetirse. Las palabras con alta probabilidad no aparecen.

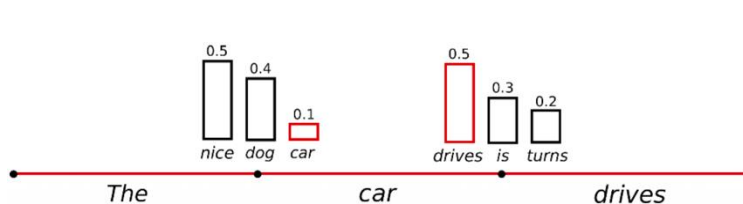


Beam search: mantiene varias secuencias posibles y selecciona la mejor a nivel global. Es computacionalmente más costosa.

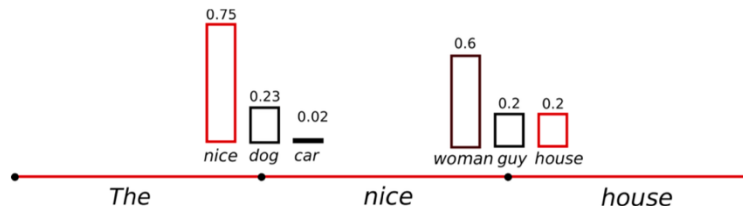


Técnicas de generación de la respuesta

Sampling: Selecciona tokens de acuerdo a una distribución de probabilidad. No es determinista. El texto parece no ser generado por un humano.



Temperatura: modifica la distribución de probabilidad haciéndola más marcada. A mayor temperatura, más diversidad (alucinación); a menor temperatura, menos diversidad (predecibles).



Resumen – Evolución de los LLMs

Etapa	Hito	Pregunta	Elemento central
IA predictiva	<= 2020	¿Qué ocurrirá o a qué categoría pertenece?	Datos y modelos entrenados
IA generativa	2020-2022	¿Qué contenido puede generar?	Modelos fundacionales
Prompt engineering	2022-2023	¿Cómo debo formular la instrucción?	Diseño del prompt
RAG y contexto aumentado	2023-2024	¿Qué información externa necesita el modelo?	Recuperación de documentos
Ingeniería de contexto	2024-2025	¿Qué información debe recibir el modelo en cada momento?	Contexto dinámico
IA agéntica	2024-2026	¿Cómo puede planificar, usar herramientas y actuar?	Agentes y flujos de trabajo
Sistemas multiagente	2025-2026	¿Cómo pueden colaborar varios agentes especializados?	Orquestación
Ingeniería de agentes	2026 =>	¿Cómo hacer que los agentes sean confiables, evaluables y gobernables?	Arquitectura, seguridad y evaluación



Cuantización y PEFT para LLMs

Motivación

El gran tamaño de los modelos de lenguaje requiere un alto volumen de memoria disponible.

La necesidad de poder desplegar y entrenar estos modelos ha llevado al desarrollo de métodos tanto para comprimir los modelos como para ajustarlos de manera eficiente.

Modelo	Parámetros	Contexto	Requerimiento de memoria
Llama 4 Scout	109B/17B	Hasta 10M	~220GB
Qwen3-235B-A22B	235B/22B	128k-256k	~470GB
Qwen3-32B	32B	128k	~64GB
Qwen3-8B	8B	128k	~16GB
Llama 3.1 405B	405B	128k	~810GB
Llama 3.1 8B	8B	128k	~16GB

https://artificialanalysis.ai/models?utm_source=chatgpt.com

Cuantización

Proceso de reducir la precisión de los valores numéricos usados para representar los parámetros del modelo.

EL proceso de cuantización genera una disminución en el tamaño del modelo al usar menos bits para representar cada parámetro y aumenta la velocidad de inferencia del modelo, debido a que los cálculos que se realizan para procesar la entrada y producir la salida pueden ser más rápidos.

Formato	Uso	Ventaja
FP32	Entrenamiento	Alta precisión
FP16	Entrenamiento/Inferencia	Menos memoria y más velocidad
BF16	Hardware moderno	Similar a FP32 con 16 bits
INT8	Inferencia eficiente	Reduce memoria
INT4	QLoRA, local LLM	Compacto

Cuantización

Postentrenamiento

La cuantización postentrenamiento es la conversión de los parámetros de un modelo entrenado a una menor precisión sin ningún preentrenamiento. Para la inferencia se decuantiza.

$$\mathbf{X}_{\text{quant}} = \text{round} \left(\frac{127}{\max |\mathbf{X}|} \cdot \mathbf{X} \right)$$

$$\mathbf{X}_{\text{dequant}} = \frac{\max |\mathbf{X}|}{127} \cdot \mathbf{X}_{\text{quant}}$$

Cuantización

Entrenamiento consciente de la cuantización

En lugar de aplicar la técnica de cuantización como un paso separado, el entrenamiento consciente de la cuantización la incorpora en el mismo proceso de entrenamiento. Este tipo de cuantización optimiza los parámetros del modelo con tal de mitigar la potencial pérdida de rendimiento asociada con la cuantización.

Fine-tuning

Es el proceso de adaptar los pesos de un modelo fundacional para que sea capaz de resolver una tarea específica.

El almacenamiento y despliegue de estos modelos adaptados para cada una de las tareas específicas se vuelven muy costosos debido a que debemos utilizar una copia de todos los parámetros para cada tarea.

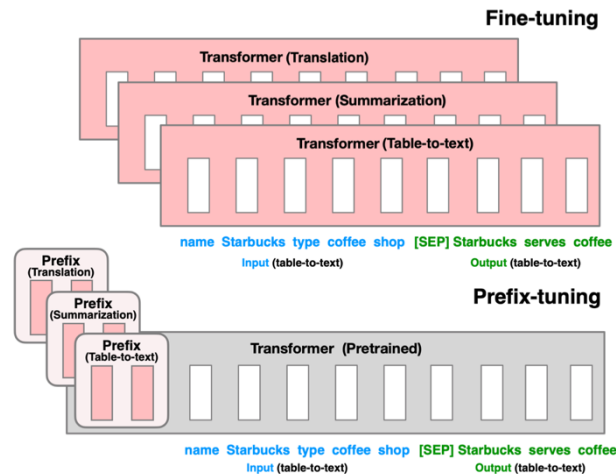
Parameter Efficient Fine-Tuning (PEFT)

Ajusta una muy pequeña proporción de los modelos fundacionales al añadir unos parámetros extra, los cuales son los que se ajustan sin adaptar el modelo.

La pequeña cantidad de parámetros ajustados son agregados sobre los parámetros del modelo fundacional, por lo que el modelo fundacional puede ser utilizado para múltiples tareas con tan solo agregar unos pocos parámetros.

Prefix tuning

Es un método aditivo, donde una secuencia de vectores continuos específicos para la tarea son agregados al inicio de la entrada. Solo los parámetros de este prefijo son optimizados y añadidos a los estados ocultos en cada una de las capas del modelo.



Parameter Efficient Fine-Tuning (PEFT)

LoRA y QLoRA: adaptación eficiente de LLMs

La idea central: en lugar de actualizar todos los parámetros del modelo, añadimos pequeñas matrices entrenables que aprenden la adaptación. El modelo base permanece congelado.

LoRA (Low-Rank Adaptation)

Añade matrices de bajo rango a las capas lineales del modelo y solo entrena esos parámetros adicionales.

1. CAPA LINEAL ORIGINAL

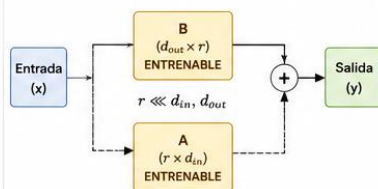
Una capa lineal usa una matriz de pesos
 $W \in \mathbb{R}^{d_{out} \times d_{in}}$



Estos son los pesos del modelo preentrenado. No se actualizan durante el fine-tuning.

2. LoRA: ADAPTACIÓN DE BAJO RANGO

Añadimos un "camino" paralelo de bajo rango que se entrena.

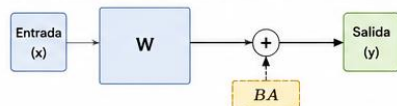


Se entrenan solo A y B (pocos parámetros). La actualización efectiva es: $\Delta W = BA$.

3. DURANTE LA INFERENCIA

El modelo usa $W + \Delta W = W + BA$.

Es equivalente a ajustar la capa con muy pocos parámetros adicionales.



VENTAJAS LoRA

- ✓ Muy pocos parámetros entrenables (miles o millones vs miles de millones).
- ✓ Menor memoria y almacenamiento.
- ✓ Entrenamiento y despliegue más rápidos.
- ✓ Fácil de combinar (múltiples LoRA para distintas tareas).

QLoRA (Quantized LoRA)

Extiende LoRA usando cuantización de 4 bits (normalmente NF4) y técnicas para entrenar modelos muy grandes con una sola GPU.

1. MODELO BASE CUANTIZADO EN 4 BITS

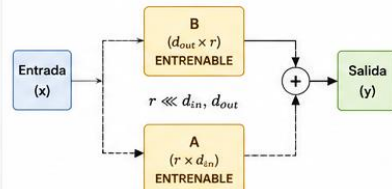
Los pesos del modelo se almacenan en 4 bits (NF4), no en 16/32 bits.



Reduce 4-8x el uso de memoria sin perder mucha calidad.

2. LoRA SOBRE PESOS CUANTIZADOS

Se añaden matrices LoRA en 16 bits (o bfloat16) y solo ellas se entrenan.



Solo se entrenan A y B en precisión alta. W permanece en 4 bits.

3. TÉCNICAS CLAVE DE QLoRA

- **Cuantización NF4:** formato de 4 bits optimizado para pesos normales.
- **Paged Optimizers:** optimizadores que usan memoria de forma eficiente (evitan picos de memoria).
- **Double Quantization:** cuantiza también las constantes de cuantización para ahorrar más memoria.

VENTAJAS QLoRA

- ✓ Permite fine-tuning de modelos muy grandes (13B, 33B, 65B, 70B, etc.) en una sola GPU.
- ✓ Menor memoria (4-bit) + pocos parámetros entrenables (LoRA).
- ✓ Rendimiento cercano al fine-tuning completo.
- ✓ Ideal cuando los recursos son limitados.



Entrenamiento y despliegue de LLMs



Motivación

Para poner en producción sistemas basados en LLMs normalmente se decide por desplegar un *servicio web* que **disponibiliza** el modelo de lenguaje a través de consultas HTTP.

Para poder desplegar estos modelos de lenguaje debemos tener grandes recursos computacionales o utilizar alguna técnica de compresión de modelos.

Computación distribuida para Deep Learning

Las GPU son dispositivos especializados que pueden realizar múltiples cálculos matemáticos. Las operaciones realizadas en DL pueden ser divididas en una serie de multiplicación de matrices, por lo que las GPU se comportan bien.

En 2012, Alex Krizhevsky e Ilya Sutskever crearon AlexNet y desde entonces, se adoptó el uso de GPUs en Deep learning.

Modelos grandes = 1 GPU es insuficiente

PARALELISMO DE LOS DATOS (Data Parallelism)

IDEA PRINCIPAL

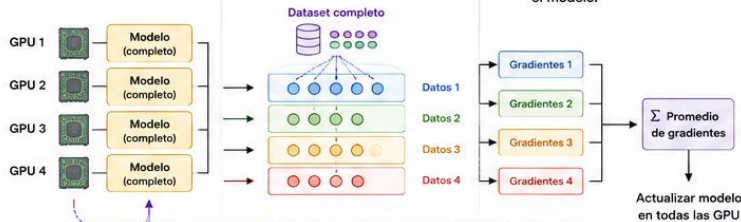
Se replica el modelo completo en cada GPU y se divide el conjunto de datos.
Cada GPU procesa diferentes datos en paralelo.

CÓMO FUNCIONA (3 ETAPAS)

1 Replicar el modelo
Cada GPU tiene una copia completa del modelo.

2 Dividir y distribuir los datos
El batch de entrenamiento se divide en partes iguales y se envía a cada GPU.

3 Agregar los resultados
Se promedian (o suman) los gradientes de todas las GPU y se actualiza el modelo.



INTUICIÓN

Todos los estudiantes (GPUs) tienen el mismo libro (modelo), pero cada uno responde preguntas diferentes (datos). Luego comparten lo aprendido (gradientes) para mejorar el libro juntos.



CARACTERÍSTICAS

- ✓ Fácil de implementar.
- ✓ Escala bien con muchos datos.
- ✓ El modelo debe caber en una sola GPU.
- ✓ Comunicación: sincronización de gradientes.

¿CUÁNDO USAR?

Cuando el modelo cabe en memoria de una GPU, pero el dataset es grande y queremos entrenar más rápido.

EJEMPLO

Entrenar ResNet-50 con ImageNet usando 8 GPUs.



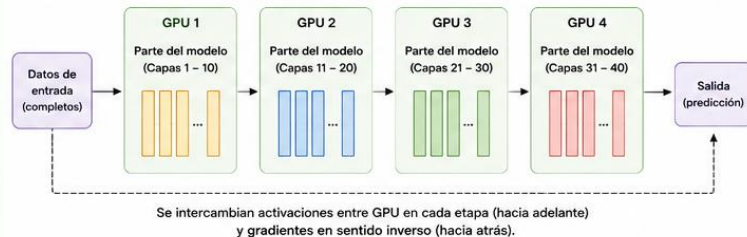
PARALELISMO DEL MODELO (Model Parallelism)

IDEA PRINCIPAL

Se divide el modelo en partes y cada GPU almacena y ejecuta solo su parte.
Permite entrenar modelos que no caben en una sola GPU.

CÓMO FUNCIONA

El modelo se particiona (por capas o por tensores) y los datos completos pasan secuencialmente por las partes del modelo.



INTUICIÓN

El libro (modelo) es muy grande para una sola persona, así que cada uno lee y trabaja solo una parte, pasándose la información.



CARACTERÍSTICAS

- ✓ Permite modelos muy grandes.
- ✓ Más complejo de implementar.
- ✓ Comunicación de activaciones (no solo gradientes).
- ✓ Puede generar cuellos de botella.

¿CUÁNDO USAR?

Cuando el modelo es tan grande que no cabe en la memoria de una sola GPU.

EJEMPLO

Entrenar un LLM de 70B parámetros dividiendo sus capas en 8 GPUs.



¿Dónde desplegar los modelos?

Herramienta	Uso	Ventajas	Desventajas
API Externa	Servicio en la nube que permite utilizar modelos mediante solicitudes desde una aplicación.	Fácil de integrar; modelos avanzados; no requiere GPUs propias.	Costo por uso; dependencia del proveedor; datos enviados a terceros; menor control.
Ollama	Permite descargar y ejecutar modelos localmente.	Instalación sencilla; funciona en CPU o GPU.	No está orientado al entrenamiento.
LM Studio	Descargar modelos, conversar con ellos y crear un servidor local.	Fácil de usar; chat integrado, API local.	Consume muchos recursos del equipo local.
Llama.cpp	Motor ligero en C/C++ para ejecutar modelos cuantizados.	Bajo consumo de memoria; soporte de cuantización.	Requiere de conocimiento experto.
vLLM	Sirve modelos en GPU con alta velocidad y atiende múltiples solicitudes.	Alto rendimiento; uso eficiente de memoria; API compatible con OpenAI.	Requiere GPU y conocimiento de configuración de servidores.
Open WebUI	Interfaz web para conversar y administrar modelos locales.	Interfaz amigable; permite conectar distintos proveedores.	No es un motor de inferencia; conexión con otra herramienta.